

Лабораторная работа №17-18. Корутины на практике: Метеосводка

Цель работы: Изучить основы корутин в Kotlin для создания асинхронных операций в Android-приложениях. Научиться использовать **launch**, **async**, диспетчеры корутин и обрабатывать длительные операции без блокировки UI-потока. Создать приложение "Weather Dashboard" с имитацией загрузки данных о погоде из разных источников.

Теоретическая часть. Что такое корутины?

Представьте, что вы готовите ужин: пока варится суп, вы можете нарезать салат, а пока запекается мясо — накрыть на стол. Вы не стоите и не ждёте, пока закончится каждая операция. **Корутины** работают так же — они позволяют вашему приложению выполнять несколько задач "параллельно", не блокируя основной поток (UI).

Зачем нужны корутины?

- Загрузка данных из сети без "зависания" приложения
- Работа с базой данных
- Обработка файлов
- Простой и читаемый код

Основные понятия

Понятие	Что это?	Аналогия
Coroutine	Легковесный поток	Задача на вашей кухне (нарезать салат)
Suspend функция	Функция, которая может "приостановиться"	Вы отошли от салата, чтобы проверить суп
Coroutine Scope	Область жизни корутины	Время приготовления ужина (когда ужин готов, все задачи завершаются)
Dispatcher	Где выполняется корутина	Какая конфорка на плите используется
Job	Контроль над корутиной	Таймер для каждого блюда

Диспетчеры (Dispatchers)

```
// Где будет выполняться ваша корутина?  
Dispatchers.Main // UI-поток (обновление экрана)  
Dispatchers.IO // Сеть, файлы, база данных  
Dispatchers.Default // Тяжёлые вычисления (сортировка больших массивов)
```

Сравнение:

Dispatcher	Когда использовать	Пример
Main	Обновление UI	Изменить текст в Text()
IO	Чтение/запись данных	Загрузка картинки из интернета
Default	Вычисления	Обработка 10000 элементов списка

launch vs async

```
// launch - запустить и забыть (ничего не возвращает)
viewModelScope.launch {
    loadData() // просто выполняется
}

// async - запустить и получить результат
val result = viewModelScope.async {
    calculateSomething() // вернёт результат
}.await()
```

Когда что использовать?

Метод	Возвращает результат?	Когда использовать
launch	Нет (возвращает Job)	Просто выполнить задачу (отправить логи)
async	Да (возвращает Deferred<T>)	Нужен результат (загрузить данные)

Пример suspend функции

```
// Обычная функция - выполняется сразу и полностью
fun fetchData(): String {
    // Если здесь долгая операция - UI зависнет!
    Thread.sleep(2000)
    return "Data"
}
```

```
// Suspend функция - может приостановиться и не блокирует поток
suspend fun fetchDataSuspend(): String {
    delay(2000) // приостановка на 2 секунды (UI не зависает!)
    return "Data"
}
```

Ключевое отличие:

- `Thread.sleep()` — блокирует поток (UI зависает ❌)
- `delay()` — приостанавливает корутину (UI работает ✅)

Практическая часть

Описание проекта

Мы создадим **Weather Dashboard** — приложение, которое показывает погоду из трёх источников:

1. 🌡️ Температура (загружается 2 секунды)
2. 💧 Влажность (загружается 1.5 секунды)
3. 🌀 Скорость ветра (загружается 1 секунда)

Особенности:

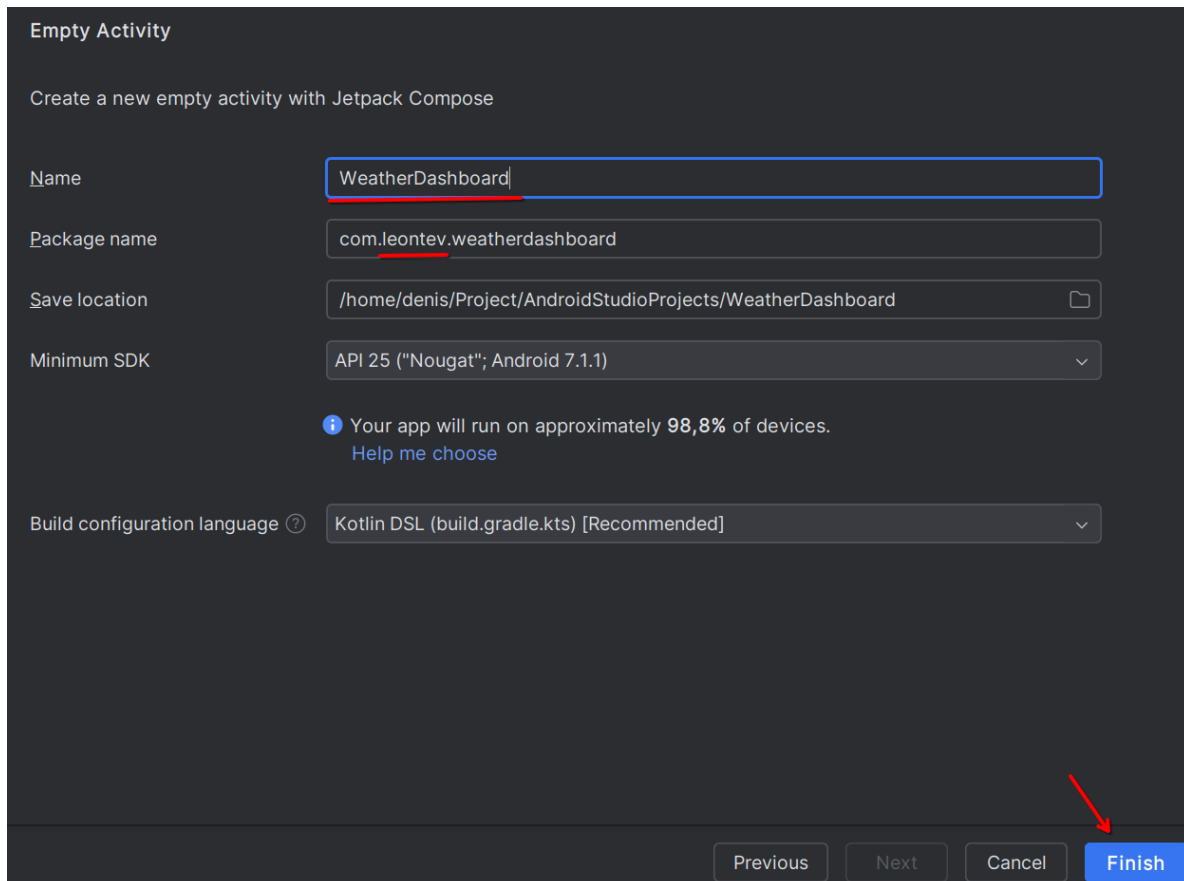
- Данные загружаются **параллельно** (не 4.5 секунды, а ~2 секунды)
- Показывается индикатор загрузки
- Кнопка "Refresh" для перезагрузки
- Обработка ошибок

Шаг 1. Создание проекта и настройка репозитория

Практическая часть. Подготовка к работе

Создание проекта

1. Откройте **Android Studio**
2. Выберите **File → New → New Project**
3. Выберите **Empty Activity → Next**



Empty Activity

Create a new empty activity with Jetpack Compose

Name: WeatherDashboard

Package name: com.leontev.weatherdashboard

Save location: /home/denis/Project/AndroidStudioProjects/WeatherDashboard

Minimum SDK: API 25 ("Nougat"; Android 7.1.1)

ⓘ Your app will run on approximately 98,8% of devices.
[Help me choose](#)

Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

Previous Next Cancel **Finish**

4. Укажите параметры:

- **Name:** WeatherDashboard
- **Package name:** com.ваше_фю.weatherdashboard
- **Minimum SDK:** API 25

5. Нажмите **Finish**

Дождитесь загрузки проекта.

Создание Git-репозитория

Перед началом работы создайте Git-репозиторий для отслеживания изменений и защиты от потери данных.

Вариант 1: Через Android Studio (рекомендуется)

1. В Android Studio выберите **VCS → Enable Version Control Integration**
2. Выберите **Git → OK**

3. Выберите **Git** → **Commit** (или **Ctrl+K**)
4. Выберите все файлы, введите сообщение: **"Initial commit"**
5. Нажмите **Commit**

Вариант 2: Через терминал

1. Откройте терминал в Android Studio (**View** → **Tool Windows** → **Terminal**)
2. Выполните команды:

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория на GitHub

1. Откройте **GitHub** и войдите в аккаунт.
2. Нажмите **New repository** (зелёная кнопка)
3. **Repository name:** **WeatherDashboard_FIO** (где FIO — ваша фамилия)
4. Выберите **Public** (публичный репозиторий)
5. **НЕ** ставьте галочки на "Add README" и ".gitignore"
6. Нажмите **Create repository**
7. Скопируйте URL репозитория

Привязка локального репозитория к удалённому

В терминале Android Studio выполните:

git branch -M main

git remote add origin <URL_репозитория>

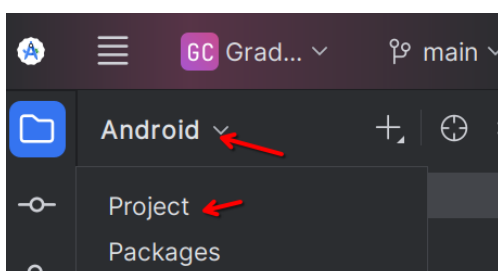
git push -u origin main

* Замените <URL_репозитория> на скопированный URL

При первом **push** вас попросят ввести логин и пароль **GitHub**.

3. Создание папки img

1. В Android Studio переключитесь в режим просмотра **Project** (в левом верхнем углу панели Project)



2. Найдите **app**
3. Кликните правой кнопкой на папке **app**
4. Выберите **New** → **Directory**
5. Введите имя: **img**
6. Нажмите **OK**

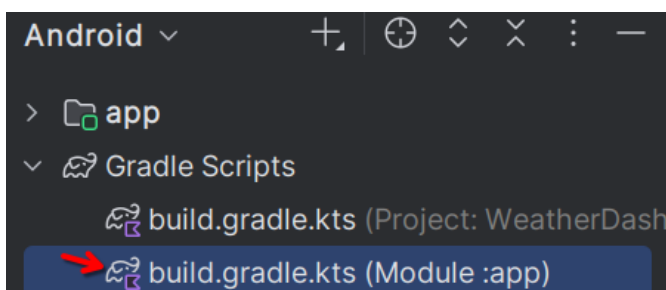
Полный путь к папке: `/app/img/`

Шаг 2. Настройка зависимостей

2.1. Добавление библиотек

Что мы делаем: Для работы с корутинами нужны специальные библиотеки. Мы добавим их в конфигурацию проекта.

1. Откройте файл `build.gradle.kts` (**Module: app**)



2. Найдите блок **dependencies**

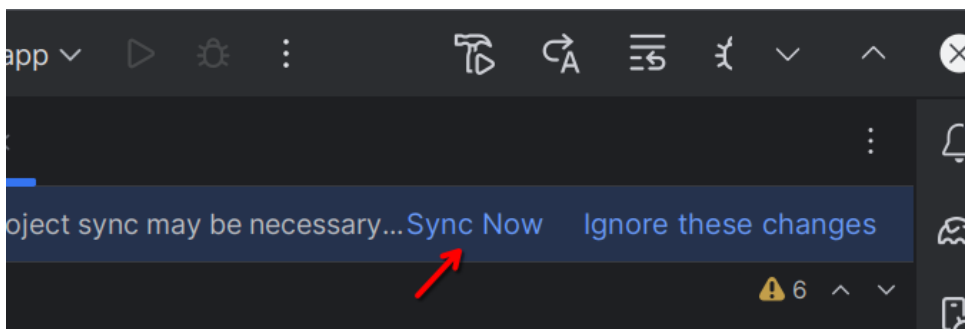
Добавьте зависимости в блок dependencies:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
```

Объяснение:

- **lifecycle-viewmodel-compose** — для создания ViewModel в Compose с поддержкой корутин
- **kotlinx-coroutines-android** — корутины для Android (включает Dispatchers.Main)
- **kotlinx-coroutines-core** — базовая библиотека корутин

3. Нажмите **Sync Now** (появится сверху файла)



Скриншот для отчёта

- Скриншот файла `build.gradle.kts` с добавленными зависимостями
- Сохраните как: `step2_dependencies_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 2: Add coroutines dependencies"`

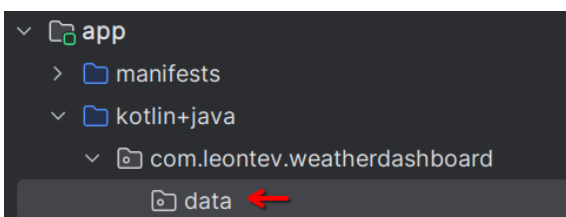
`git push`

Шаг 3. Создание data-модели

3.1. Создание пакета data

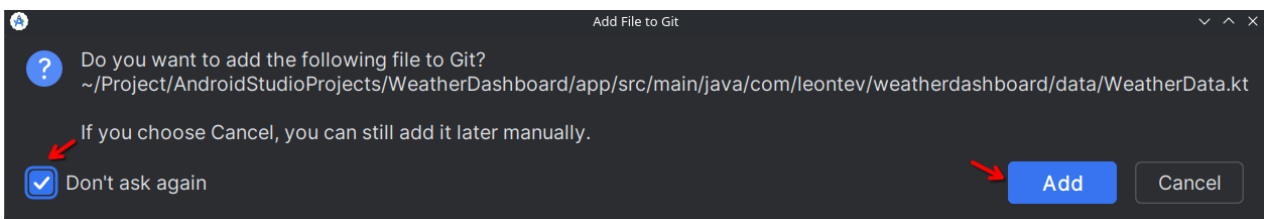
Создадим структуру для хранения данных о погоде.

1. В режиме **Android** найдите пакет `com.ваше_фio.weatherdashboard`
2. Правой кнопкой мыши → **New** → **Package**
3. Введите: **data**
4. Нажмите **OK**



3.2. Создание data class

1. Создайте файл `WeatherData.kt` в пакете `data`
2. Соглашаемся на отслеживание файла в Git:



3. Откройте файл `WeatherData.kt` и добавьте код:

```
data class WeatherData(  
    val temperature: Int? = null,  
    val humidity: Int? = null,  
    val windSpeed: Int? = null,  
    val isLoading: Boolean = false,  
    val error: String? = null  
)
```

- **data class** — специальный класс для хранения данных (автоматически создаёт equals(), hashCode(), copy())
- **temperature: Int? = null** — температура, может быть null (пока не загружена)
- **humidity: Int? = null** — влажность
- **windSpeed: Int? = null** — скорость ветра
- **isLoading: Boolean = false** — флаг загрузки (показывать ли индикатор)
- **error: String? = null** — текст ошибки (если произошла)

Зачем ? и = null?

- **?** — тип может быть null (nullable type)
- **= null** — значение по умолчанию (при создании объекта можно не указывать)

Скриншоты для отчёта

- Скриншот кода **WeatherData.kt**
- Скриншот структуры проекта с пакетом **data**
- Сохраните как: **step3_weather_data_FIO.png** и **step3_data_package_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 3: Create WeatherData model"

git push

Шаг 4. Создание Repository (имитация API)

4.1. Создание файла WeatherRepository

Создадим "хранилище данных", которое будет имитировать загрузку данных о погоде с сервера. В реальном приложении здесь были бы запросы к API.

1. В пакете **data** создайте файл **WeatherRepository.kt**
2. Откройте файл **WeatherRepository.kt**
3. Добавим **необходимые импорты**:

```
import kotlinx.coroutines.delay
import kotlin.random.Random
```

4. Далее создадим класс **WeatherRepository**:

```
class WeatherRepository {
}
```

Внутри класса **WeatherRepository** создадим **3 функции**:

- Имитация загрузки температуры (2 секунды)

```
suspend fun fetchTemperature(): Int {  
    delay(2000) // приостановка корутины на 2 секунды  
    return Random.nextInt(15, 35) // случайное число от 15 до 35  
}
```

- Имитация загрузки влажности (1.5 секунды)

```
suspend fun fetchHumidity(): Int {  
    delay(1500)  
    return Random.nextInt(40, 80) // от 40% до 80%  
}
```

- Имитация загрузки скорости ветра (1 секунда)

```
suspend fun fetchWindSpeed(): Int {  
    delay(1000)  
    return Random.nextInt(0, 20) // от 0 до 20 м/с  
}
```

- **suspend fun** — функция может приостанавливаться (вызываться только из корутины)
- **delay(2000)** — приостановка на 2000 миллисекунд (2 секунды), **не блокирует поток**
- **Random.nextInt(15, 35)** — случайное целое число от 15 до 34 (имитация реальных данных)
- Каждая функция возвращает **Int** — результат "загрузки"

Скриншоты для отчёта

- Скриншот кода **WeatherRepository.kt**
- Сохраните как: **step4_repository_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 4: Create WeatherRepository with suspend functions"

git push

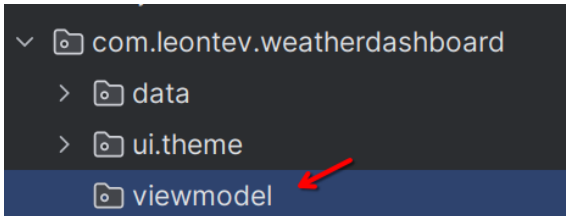
Шаг 5. Создание ViewModel

На этом шаге мы начнём работать с архитектурным компонентом **ViewModel**. Он будет отвечать за состояние экрана погоды и за загрузку данных.

5.1. Создание пакета viewmodel

Для начала создадим отдельный пакет, в котором будут храниться все ViewModel.

1. В режиме **Android** найдите пакет **com.ваше_фio.weatherdashboard**
2. Правой кнопкой мыши → **New** → **Package**
3. Введите: **viewmodel**
4. Нажмите **OK**



5. Внутри **viewmodel** создайте файл **WeatherViewModel.kt**

Именно в нём мы будем описывать логику управления состоянием экрана.

5.2. Создание класса ViewModel

ViewModel будет:

- хранить данные погоды,
- управлять состоянием UI,
- запускать корутины для загрузки данных.

Откройте файл **WeatherViewModel.kt**

1. Добавьте необходимые импорты (обратите внимание **на ваши пакеты**):

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.ваш_пакет.weatherdashboard.data.WeatherData
import com.ваш_пакет.weatherdashboard.data.WeatherRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
```

Они понадобятся нам для работы с **ViewModel**, **корутинами** и **StateFlow**.

2. Далее создаём класс **WeatherViewModel**, который наследуется от **ViewModel**:

```
class WeatherViewModel : ViewModel() { }
```

Так мы получаем компонент, который переживает пересоздание Activity и не теряет данные при повороте экрана.

Далее весь код пишем внутри класса

Внутри класса первым делом создаём **приватное поле репозитория**.

```
private val repository = WeatherRepository()
```

Репозиторий будет отвечать за получение данных, а **ViewModel** — за работу с ними. После этого создаём **приватное изменяемое состояние** `_weatherState`.

```
private val _weatherState = MutableStateFlow(WeatherData())
```

Оно будет хранить текущие данные погоды и может изменяться только внутри **ViewModel**.

Чтобы UI мог получать данные, но не менять их напрямую, создаём **публичное неизменяемое состояние** `weatherState`.

```
val weatherState: StateFlow<WeatherData> = _weatherState.asStateFlow()
```

Оно основано на `_weatherState`, но доступно только для чтения.

Затем добавляем **внутри класса** блок `init`.

```
init {  
    loadWeatherData()  
}
```

Он выполняется сразу после создания **ViewModel**, поэтому именно здесь мы запускаем первую загрузку данных.

Внутри класса также создаём функцию `loadWeatherData()`.

```
fun loadWeatherData() {  
    // TODO: здесь будет логика загрузки  
}
```

Пока она будет пустой — позже в ней появится логика загрузки данных из репозитория с использованием корутин.

В результате мы получили **ViewModel**, которая:

- хранит состояние экрана,
- инкапсулирует логику работы с данными,
- сразу после создания начинает загрузку информации.

Пояснения:

- **ViewModel** — базовый класс, который сохраняет данные при пересоздании **Activity**
- **viewModelScope** — область жизни корутин, связанная с **ViewModel**

- **MutableStateFlow** — изменяемый поток данных, доступный только внутри ViewModel
- **StateFlow** — неизменяемый поток, который используется в UI
- **asStateFlow()** — превращает изменяемый поток в read-only
- **init { }** — блок, который выполняется при создании ViewModel

Зачем использовать два поля (**_weatherState** и **weatherState**)?

Это сделано для соблюдения принципа инкапсуляции:

- **_weatherState** — приватное, ViewModel может изменять данные
- **weatherState** — публичное, UI может только читать состояние, но не менять его напрямую

5.3. Добавление метода последовательной загрузки

На этом шаге мы реализуем **последовательную загрузку данных**. Это заведомо **не самый эффективный вариант**, но он нужен нам для понимания проблемы и последующего сравнения с параллельной загрузкой.

Идея простая: мы будем загружать данные **по очереди**, один запрос за другим, и обновлять состояние экрана после каждого шага.

Обновление метода **loadWeatherData()**

Доработаем функцию **loadWeatherData()**. Первое, что мы делаем — запускаем корутину в **viewModelScope**. Так вся загрузка будет привязана к жизненному циклу ViewModel и автоматически отменится, если ViewModel будет уничтожена.

```
fun loadWeatherData() {
    viewModelScope.launch {
        // Начинаем загрузку
        _weatherState.value = _weatherState.value.copy(
            isLoading = true,
            error = null
        )
    }
}
```

В самом начале загрузки обновляем состояние:

- включаем индикатор загрузки,
- сбрасываем возможную предыдущую ошибку.

После этого переходим к получению данных:

```

fun loadWeatherData() {
    viewModelScope.launch {
        // Начинаем загрузку
        _weatherState.value = _weatherState.value.copy(...)
        // Загружаем по очереди (медленно!)
        val temperature = repository.fetchTemperature() // 2 сек
        _weatherState.value = _weatherState.value.copy(temperature = temperature)
        val humidity = repository.fetchHumidity() // + 1.5 сек
        _weatherState.value = _weatherState.value.copy(humidity = humidity)
        val windSpeed = repository.fetchWindSpeed() // + 1 сек
        _weatherState.value = _weatherState.value.copy(windSpeed = windSpeed)
        // Всё загружено
        _weatherState.value = _weatherState.value.copy(isLoading = false)
    }
}

```

Мы загружаем их **последовательно**:

1. Сначала получаем температуру. После получения сразу обновляем состояние, чтобы UI мог отобразить результат.
2. Затем запрашиваем влажность и снова обновляем состояние.
3. В конце загружаем скорость ветра и также сохраняем её в состоянии.

Только после того как все данные получены, мы выключаем состояние загрузки.

Всю логику оборачиваем в try-catch, чтобы обработать возможные ошибки:

```

fun loadWeatherData() {
    viewModelScope.launch {
        // Начинаем загрузку
        _weatherState.value = _weatherState.value.copy(...)
        try {
            // Загружаем по очереди (медленно!)
            val temperature = repository.fetchTemperature() // 2 сек
            _weatherState.value = _weatherState.value.copy(temperature = temperature)
            val humidity = repository.fetchHumidity() // + 1.5 сек
            _weatherState.value = _weatherState.value.copy(humidity = humidity)
            val windSpeed = repository.fetchWindSpeed() // + 1 сек
            _weatherState.value = _weatherState.value.copy(windSpeed = windSpeed)
            // Всё загружено
            _weatherState.value = _weatherState.value.copy(isLoading = false)
        } catch (e: Exception) {
            _weatherState.value = _weatherState.value.copy(
                isLoading = false,
                error = "Ошибка загрузки: ${e.message}"
            )
        }
    }
}

```

Если во время загрузки что-то пойдёт не так, мы:

- выключаем индикатор загрузки,

- сохраняем текст ошибки в состоянии, чтобы UI мог его отобразить.

Объяснение:

- `viewModelScope.launch { }` — запускаем корутину внутри ViewModel
- `_weatherState.value` — текущее состояние экрана
- `copy(...)` — создаём новое состояние, изменяя только нужные поля
- `try-catch` — защищаемся от ошибок во время загрузки данных

Почему этот способ плохой

Все запросы выполняются **строго по очереди**:

- температура — ~2 секунды
- влажность — ещё ~1.5 секунды
- ветер — ещё ~1 секунда

В сумме пользователь ждёт около **4.5 секунд**, хотя данные могли бы загружаться одновременно.

Важно: Это **неэффективный** способ! Мы исправим его позже.

Скриншоты для отчёта

- Скриншот кода `WeatherViewModel.kt` с методом `loadWeatherData()`
- Сохраните как: `step5_viewmodel_sequential_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 5: Create ViewModel with sequential loading"`

`git push`

Шаг 6. Создание UI с Jetpack Compose

6.1. Открытие MainActivity

Откройте `MainActivity.kt`. Удалите неиспользуемый код и обновите класс:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            WeatherDashboardTheme {

            }
        }
    }
}
```

6.2. Создание основного экрана

Добавьте функцию `WeatherDashboardScreen()` в конец файла `MainActivity.kt`:

```
@Composable
fun WeatherDashboardScreen(
    viewModel: WeatherViewModel = viewModel()
) {
    val weatherState by viewModel.weatherState.collectAsState()
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "☀️ Weather Dashboard",
            style = MaterialTheme.typography.headlineLarge
        )
        Spacer(modifier = Modifier.height(32.dp))
        // TODO: здесь будут карточки с данными
        Spacer(modifier = Modifier.height(32.dp))
        // TODO: здесь будет кнопка обновления
    }
}
```

Эмодзи для вставки: солнце за облаками - ☀️

Можете сразу **обновить импорты**, на те, которые нам потребуются для написания кода (обратите внимание на **ваши пакеты**):

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
```

```
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.ваш_пакет.weatherdashboard.ui.theme.WeatherDashboardTheme
import com.ваш_пакет.weatherdashboard.viewmodel.WeatherViewModel
```

Объяснение:

- `@Composable` — функция описывает UI
- `viewModel: WeatherViewModel = viewModel()` — получаем ViewModel (создаётся автоматически)
- `collectAsState()` — подписываемся на изменения StateFlow, при изменении UI перерисовуется
- `by` — делегат для автоматической распаковки State
- `Column` — вертикальный контейнер
- `fillMaxSize()` — занять весь экран
- `padding(16.dp)` — отступы по краям
- `Spacer(height = 32.dp)` — пустое пространство между элементами

6.3. Создание карточки данных

На этом шаге мы создаём универсальную карточку, которая будет использоваться для отображения погодных параметров: температуры, влажности и скорости ветра.

Идея простая: вместо того чтобы каждый раз заново верстать одинаковые элементы, мы выносим их в отдельный `@Composable` и переиспользуем.

Добавьте функцию `WeatherCard()` в конец файла:

```
@Composable
fun WeatherCard(
    emoji: String,
    title: String,
    value: String,
    isLoading: Boolean
) { }
```

WeatherCard — это **composable-функция**, которая отвечает за внешний вид одной карточки с данными.

Она принимает на вход:

- **emoji** — иконку в виде эмодзи, чтобы визуально обозначить тип данных
- **title** — название параметра (Температура, Влажность и т.д.)
- **value** — текущее значение параметра
- **isLoading** — флаг, показывающий, загружаются ли данные

Таким образом, одна и та же карточка может:

- либо показывать данные,
- либо отображать индикатор загрузки.

Создайте внутри следующую содержимое:

```
@Composable
fun WeatherCard(
    emoji: String,
    title: String,
    value: String,
    isLoading: Boolean
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        1 Row(...) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                2 Text(...)
                Spacer(modifier = Modifier.width(16.dp))
                3 Text(...)
            }
            4 if (isLoading) {...} else {
                5 Text(...)
            }
        }
    }
}
```

Допишите в цифры код:

1.

```
Row(  
    modifier = Modifier  
        .padding(16.dp)  
        .fillMaxWidth(),  
    verticalAlignment = Alignment.CenterVertically,  
    horizontalArrangement = Arrangement.SpaceBetween  
) {
```

2.

```
Text(  
    text = emoji,  
    style = MaterialTheme.typography.headlineMedium  
)
```

3.

```
Text(  
    text = title,  
    style = MaterialTheme.typography.titleLarge  
)
```

4.

```
if (isLoading) {  
    CircularProgressIndicator(  
        modifier = Modifier.size(24.dp),  
        strokeWidth = 2.dp  
    )  
} else {
```

5.

```
Text(  
    text = value,  
    style = MaterialTheme.typography.titleLarge,  
    color = MaterialTheme.colorScheme.primary  
)
```

Внутреннее содержимое: Внутри карточки используется Row, который делит содержимое на две части:

Левая часть:

- эмодзи (визуальный акцент),
- название параметра.

Они располагаются горизонтально и выровнены по центру по вертикали.

Правая часть:

- если данные ещё загружаются, показывается CircularProgressIndicator,
- если загрузка завершена — отображается значение параметра.

Таким образом пользователь сразу видит:

- либо процесс загрузки,
- либо уже готовый результат.

Почему это удобно

- Карточка универсальная и переиспользуемая
- UI автоматически реагирует на состояние isLoading
- Код экрана становится чище и понятнее
- Легко добавить новые параметры погоды без дублирования верстки

6.4. Структура MainActivity

Измените код внутри MainActivity:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            WeatherDashboardTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    WeatherDashboardScreen()
                }
            }
        }
    }
}
```

- `WeatherDashboardTheme` — тема приложения (создаётся автоматически)
- `Surface` — контейнер с фоном
- `WeatherDashboardScreen()` — наш главный экран (создадим дальше)

6.5. Добавление карточек на экран

Замените `// TODO:` здесь будут карточки с данными в функции

`WeatherDashboardScreen()`. Добавляем 3 карточки

```
Spacer(modifier = Modifier.height(32.dp))
```

```
// TODO: здесь будут карточки с данными
```

ЗАМЕНА

Эмодзи для вставки:

- термометр - 🌡️
- капля - 💧
- торнадо - 🌪️

1. Карточка для температуры:

```
WeatherCard(
    emoji = "🌡️",
    title = "Temperature",
    value = weatherState.temperature?.let { "$it°C" } ?: "-",
    isLoading = weatherState.isLoading && weatherState.temperature == null
)
```

2. Карточка для влажности:

```
WeatherCard(
    emoji = "💧",
    title = "Humidity",
    value = weatherState.humidity?.let { "$it%" } ?: "-",
    isLoading = weatherState.isLoading && weatherState.humidity == null
)
```

3. Карточка для ветра:

```
WeatherCard(
    emoji = "🌪️",
    title = "Wind Speed",
    value = weatherState.windSpeed?.let { "$it m/s" } ?: "-",
    isLoading = weatherState.isLoading && weatherState.windSpeed == null
)
```

- `weatherState.temperature` — получаем значение из `StateFlow`
- `?.let { "$it°C" }` — если не `null`, форматируем (например, "25°C")

- `?: "—"` — если null, показываем прочерк
- `isLoading && temperature == null` — показываем индикатор, только если данных ещё нет

6.6. Добавление кнопки обновления

Замените `// TODO: здесь будет кнопка обновления` в функции `WeatherDashboardScreen()`.

```
Spacer(modifier = Modifier.height(32.dp))
// TODO: здесь будет кнопка обновления
```

ЗАМЕНА

Эмодзи для вставки:

против часовой стрелки - 

```
Button(
    onClick = { viewModel.loadWeatherData() },
    enabled = !weatherState.isLoading
) {
    Text(text = if (weatherState.isLoading) "Loading..." else "🔄 Refresh Weather")
}

if (weatherState.error != null) {
    Spacer(modifier = Modifier.height(16.dp))
    Text(
        text = weatherState.error!!,
        color = MaterialTheme.colorScheme.error,
        style = MaterialTheme.typography.bodyMedium
    )
}
```

- `onClick = { viewModel.loadWeatherData() }` — при нажатии запускаем загрузку
- `enabled = !weatherState.isLoading` — кнопка неактивна во время загрузки
- `if (weatherState.error != null)` — показываем ошибку, если есть
- `!!` — force unwrap (мы проверили, что не null)

Скриншоты для отчёта

- Скриншот полного кода `MainActivity.kt`
- Скриншот приложения в эмуляторе (с индикаторами загрузки)
- Скриншот приложения после загрузки (с данными)

Сохраните как:

- `step6_mainactivity_code_FIO.png`
- `step6_loading_FIO.png`
- `step6_loaded_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 6: Create UI with Compose"`

`git push`

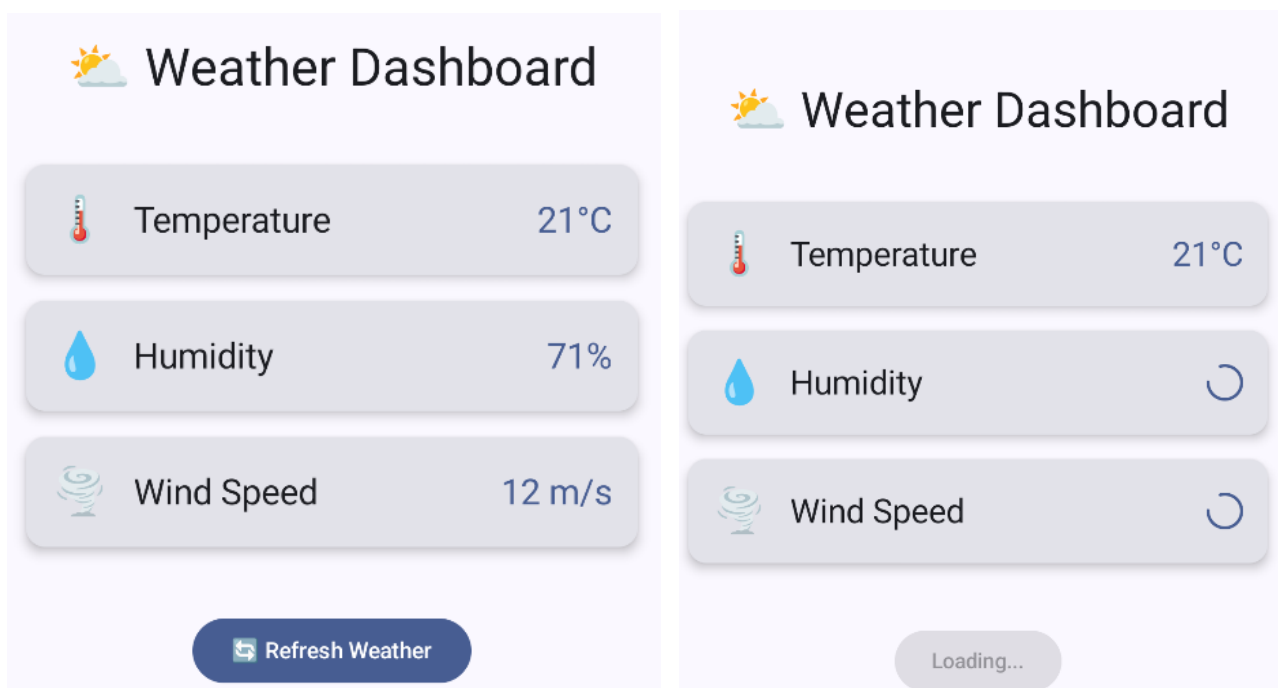
Шаг 7. Запуск и тестирование последовательной загрузки

7.1. Запуск приложения

1. Нажмите  **Run 'app'** в Android Studio
2. Выберите эмулятор или подключённое устройство
3. Дождитесь установки приложения

Что вы увидите:

1. При запуске — три карточки с индикаторами загрузки
2. Через 2 секунды — появится температура (остальные всё ещё загружаются)
3. Через 3.5 секунды — появится влажность
4. Через 4.5 секунды — появится скорость ветра



Проблема: Загрузка занимает **4.5 секунды**, хотя данные не зависят друг от друга!

7.2. Засеките время

1. Нажмите кнопку " Refresh Weather"
2. Используйте секундомер на телефоне
3. Запишите время от нажатия до полной загрузки

Ожидаемое время: ~4.5 секунды

Скриншоты для отчёта

- Скриншот процесса загрузки (частично загруженные данные)
- Скриншот после полной загрузки

Сохраните как:

- **step7_sequential_loading_FIO.png**
- **step7_sequential_done_FIO.png**

Коммит пока не нужен (код не изменился)

Шаг 8. Оптимизация: параллельная загрузка с async

8.1. Что мы меняем?

Сейчас загрузка идёт **последовательно**:

Температура (2 сек) → Влажность (1.5 сек) → Ветер (1 сек) = 4.5 сек

Мы сделаем **параллельную** загрузку:

Температура (2 сек) ⊣

Влажность (1.5 сек) | — → Всё готово за 2 сек!

Ветер (1 сек) ⊢

8.2. Изменение loadWeatherData()

Откройте **WeatherViewModel.kt** и добавьте импорт:

```
import kotlinx.coroutines.async
```

Замените внутри метода **loadWeatherData()** блока **try** весь код:

```
try {  
    // Запускаем ВСЕ запросы параллельно!  
    val temperatureDeferred = async { repository.fetchTemperature() }  
    + val humidityDeferred = async { repository.fetchHumidity() }  
    val windSpeedDeferred = async { repository.fetchWindSpeed() }
```

Продолжаем писать внутри блока **try**:

```
// Ждём результатов (они уже загружаются параллельно)
val temperature = temperatureDeferred.await()
+ val humidity = humidityDeferred.await()
val windSpeed = windSpeedDeferred.await()
```

И последнее что мы добавим:

```
// Обновляем состояние одним разом
_weatherState.value = WeatherData(
    temperature = temperature,
    humidity = humidity,
    windSpeed = windSpeed,
    isLoading = false,
    error = null
)
} catch (e: Exception) {
```

- `async { }` — запускает корутину и **сразу возвращает** `Deferred<T>` (обещание результата)
- Три `async` запускаются **мгновенно** (не ждут друг друга)
- `await()` — ждёт результат конкретного `async`
- Общее время = время самой медленной операции (2 секунды вместо 4.5!)

Как это работает:

// t=0: Все три запускаются одновременно

val temp = async { ... } // начинает работу

val hum = async { ... } // начинает работу

val wind = async { ... } // начинает работу

// t=2000: ждём самый медленный (температуру)

val t = temp.await() // готово сразу (уже загружено)

val h = hum.await() // готово сразу (закончилось раньше)

val w = wind.await() // готово сразу (закончилось раньше)

8.3. Сравнение: до и после

Подход	Код	Время
Последовательный	<code>val temp = fetch()</code>	4.5 сек
Параллельный	<code>val temp = async { fetch() }.await()</code>	2 сек

Когда использовать `async`?

-  Несколько независимых запросов (загрузка данных с разных API)
-  Нужны результаты всех операций
-  Операции зависят друг от друга (сначала логин, потом загрузка профиля)

Скриншоты для отчёта

- Скриншот нового кода `loadWeatherData()` с `async`
- Сохраните как: `step8_parallel_code_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 8: Implement parallel loading with async"`

`git push`

Шаг 9. Тестирование параллельной загрузки

9.1. Запуск и сравнение

1. Запустите приложение заново
2. Нажмите " Refresh Weather"
3. Засеките время

Ожидаемое время: ~2 секунды (вместо 4.5!)

9.2. Наблюдения

Что изменилось:

- Все три индикатора крутятся **одновременно**
- Все три значения появляются **почти одновременно** (~2 сек)
- Приложение стало **в 2.25 раза быстрее!**

Почему не ровно 2.0 секунды?

- Небольшая задержка на обработку результатов
- Время на обновление UI
- Погрешности эмулятора

Скриншоты для отчёта

- Скриншот процесса загрузки (все индикаторы крутятся)
- Скриншот после загрузки с **засечённым временем** (можно добавить текст поверх скриншота)

Сохраните как:

- `step9_parallel_loading_FIO.png`
- `step9_parallel_time_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 9: Test parallel loading performance"`

`git push`

Шаг 10. Добавление загрузки с показом прогресса

10.1. Обновление WeatherData

Добавим возможность показывать, какие данные уже загружены, а какие ещё нет.

Откройте `data/WeatherData.kt` и добавьте новое поле (не забудьте запятую):

```
val error: String? = null,  
+val loadingProgress: String = ""
```

- `loadingProgress: String = ""` — текстовый индикатор загрузки (например, "Загружается температура...")

10.2. Обновление ViewModel

Откройте `viewmodel/WeatherViewModel.kt` и добавьте в метод `loadWeatherData()`

запуск загрузки:

```
fun loadWeatherData() {  
    viewModelScope.launch {  
        // Начинаем загрузку  
        _weatherState.value = _weatherState.value.copy(  
            isLoading = true,  
            error = null,  
            +loadingProgress = "Запуск загрузки..."  
        )  
    }  
}
```

В начале блока **try** обновим прогресс:

```
try {  
    // Обновляем прогресс  
    _weatherState.value = _weatherState.value.copy(  
    +    loadingProgress = "Загружаем температуру, влажность, скорость ветра..."  
    )  
}
```

В **_weatherState.value** добавим отображение загрузки:

```
_weatherState.value = WeatherData(  
    temperature = temperature,  
    humidity = humidity,  
    windSpeed = windSpeed,  
    isLoading = false,  
    error = null,  
    + loadingProgress = "Загрузка завершена!"  
)
```

В блоке **catch** добавим пустую загрузку:

```
} catch (e: Exception) {  
    _weatherState.value = _weatherState.value.copy(  
        isLoading = false,  
        error = "Ошибка загрузки: ${e.message}",  
        + loadingProgress = ""  
    )  
}
```

10.3. Отображение прогресса в UI

Откройте **MainActivity.kt** и в функции **WeatherDashboardScreen()** добавьте после кнопки **Button**:

```
Button(...) {...}  
+ if (weatherState.loadingProgress.isNotEmpty()) {  
    Spacer(modifier = Modifier.height(8.dp))  
    Text(  
        text = weatherState.loadingProgress,  
        style = MaterialTheme.typography.bodySmall,  
        color = MaterialTheme.colorScheme.secondary  
    )  
+ }
```

- `isNotEmpty()` — показываем только если есть текст

- `bodySmall` — маленький шрифт
- `secondary` — неярко́й цвет (не отвлекает)

Скриншоты для отчёта

- Скриншот приложения с текстом прогресса загрузки

Сохраните как:

- `step10_progress_indicator_FIO.png`

Выполните в терминале:

`git add .`

`git commit -m "Step 10: Add loading progress indicator"`

`git push`

Шаг 11. Добавление обработки ошибок с повтором

11.1. Создание функции с искусственной ошибкой

Добавим возможность имитировать ошибку загрузки для тестирования.

Откройте `data/WeatherRepository.kt` и добавьте в начало класса флаг для имитации ошибки:

```
class WeatherRepository {
    +private var shouldFail = false
```

Внутри класса добавьте функцию `toggleErrorSimulation()`, которая будет работать как переключатель ошибки:

```
fun toggleErrorSimulation() {
    shouldFail = !shouldFail
}
```

Измените функцию `fetchTemperature()`:

```
suspend fun fetchTemperature(): Int {
    delay(2000)
    if (shouldFail) {
        throw Exception("Сервер недоступен")
    }
    return Random.nextInt(15, 35)
}
```

- `throw Exception()` — выбрасываем исключение (попадёт в `catch` в `ViewModel`)
- `toggleErrorSimulation()` — переключатель ошибки

11.2. Добавление кнопки "Simulate Error"

Откройте **MainActivity.kt** и добавьте в функцию **WeatherDashboardScreen()** после кнопки **Button**:

```
Button(...) {...}  
Spacer(modifier = Modifier.height(8.dp))  
  
OutlinedButton(  
    onClick = {  
        // TODO: вызов метода переключения ошибки  
    }  
) {  
    Text(text = "⚠ Simulate Error")  
}
```

Эмодзи для вставки: значок с предупреждением - ⚠

Необходимый импорт:

```
import androidx.compose.material3.OutlinedButton
```

Проблема: **ViewModel** не знает о **Repository** напрямую из UI!

11.3. Добавление метода в ViewModel

Откройте **viewmodel/WeatherViewModel.kt** и добавьте внутрь класса **WeatherViewModel** новую функцию:

```
fun toggleErrorSimulation() {  
    repository.toggleErrorSimulation()  
}
```

11.4. Подключение кнопки

Измените **onClick** кнопки в **MainActivity.kt**:

```
OutlinedButton(  
    onClick = {  
        // TODO: вызов метода переключения ошибки  
    }  
) {  
    { viewModel.toggleErrorSimulation() }}
```

Итог:

```
OutlinedButton(  
    onClick = { viewModel.toggleErrorSimulation() }  
) {  
    Text(text = "⚠ Simulate Error")  
}
```

11.5.1. Первая попытка запуска

- 

Weather Dashboard



Temperature

30°C



Humidity

72%



Wind Speed

19 m/s

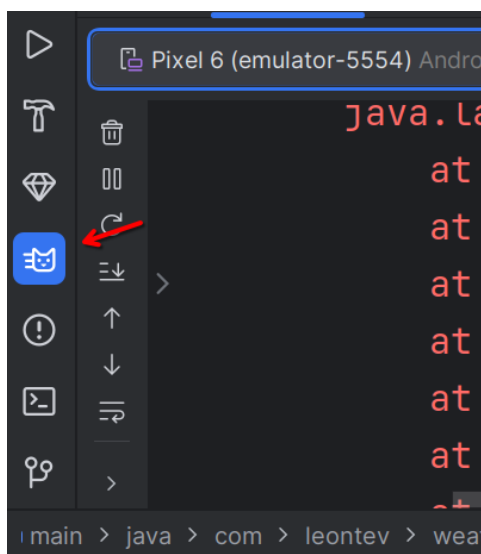
 Refresh Weather

 Simulate Error

Загрузка завершена!

11.5.2. Анализ проблемы через Logcat

1. Откройте вкладку **Logcat** внизу Android Studio



2. Найдите красную запись с текстом **FATAL EXCEPTION: main**
3. Изучите **stack trace** (стек вызовов):

```

E FATAL EXCEPTION: main
Process: com.leontev.weatherdashboard, PID: 5645
java.lang.Exception: Сервер недоступен
    at com.leontev.weatherdashboard.data.WeatherRepository.fetchTemperature(WeatherRepository.kt:17)
    at com.leontev.weatherdashboard.data.WeatherRepository$fetchTemperature$1.invokeSuspend(Unknown Source:14)
    at kotlin.coroutines.jvm.internal.BaseContinuationImpl.resumeWith(ContinuationImpl.kt:33) <7 internal lines>
    at android.os.Handler.handleCallback(Handler.java:995)
    at android.os.Handler.dispatchMessage(Handler.java:103)
    at android.os.Looper.loopOnce(Looper.java:248)
    at android.os.Looper.loop(Looper.java:338)
    at android.app.ActivityThread.main(ActivityThread.java:9067) <1 internal line>
    at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:593)
    at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:932)
Suppressed: kotlinx.coroutines.internal.DiagnosticCoroutineContextException: [StandaloneCoroutine{Cancelling}]

```

Анализ лога:

Что говорит лог	Что это значит
FATAL EXCEPTION: main	Необработанное исключение в главном потоке
java.lang.Exception: Сервер недоступен	Наше исключение из fetchTemperature()
at WeatherRepository.fetchTemperature(WeatherRepository.kt:17)	Ошибка в строке 17 файла WeatherRepository
StandaloneCoroutine{Cancelling}	Корутина была отменена из-за ошибки

Почему так произошло?

Давайте посмотрим на наш код в **WeatherViewModel.kt**:

```

try {
    val temperatureDeferred = async { repository.fetchTemperature() } // ← Здесь!
    val humidityDeferred = async { repository.fetchHumidity() }
    val windSpeedDeferred = async { repository.fetchWindSpeed() }

    val temperature = temperatureDeferred.await() // await() никогда не вызовется!
    ...
} catch (e: Exception) {
    // Сюда не попадём!
}

```

Проблема: Когда `async` запускается, он создаёт дочернюю корутину. Если в дочерней корутине возникает исключение до вызова `await()`, оно немедленно всплывает в родительскую корутину и крашит приложение, минуя наш `catch`.

11.5.3. Исправление ошибки

Откройте **WeatherViewModel.kt**

Есть два способа исправить это:

Способ 1: SupervisorJob (правильный для production)

Добавьте SupervisorJob() в viewModelScope, чтобы ошибки дочерних корутин не крашили родителя.

НО для учебных целей мы используем **Способ 3!**

Способ 2: Обработка ошибок в каждом async

Обернуть каждый **async** в **try-catch** внутри:

- Каждый **async { }** будет иметь свой **try-catch**
- Исключение не всплывает немедленно, а ждёт **await()**
- Когда вызывается **temperatureDeferred.await()**, исключение пробрасывается в **внешний catch**
- Приложение не крашится, ошибка обрабатывается корректно

Способ 3: Альтернативное решение (более элегантное):

Вместо **обёртывания** каждого **async**, можно использовать **coroutineScope**.

Обновите блок **try/catch**:

```
try {
    coroutineScope { // ← Создаём scope, который НЕ отменяет родителя при ошибке
        val tempDeferred = async { repository.fetchTemperature() }
        val humDeferred = async { repository.fetchHumidity() }
        val windDeferred = async { repository.fetchWindSpeed() }

        val temperature = tempDeferred.await()
        val humidity = humDeferred.await()
        val windSpeed = windDeferred.await()

        _weatherState.value = WeatherData(
            temperature = temperature,
            humidity = humidity,
            windSpeed = windSpeed,
            isLoading = false,
            error = null,
            loadingProgress = "Загрузка завершена!"
        )
    }
} catch (e: Exception) {
    _weatherState.value = _weatherState.value.copy(
        isLoading = false,
        error = "Ошибка загрузки: ${e.message}",
        loadingProgress = ""
    )
}
```

Что делает coroutineScope:

- Создаёт новый scope для дочерних корутин

- Ждёт завершения всех дочерних корутин
- Если хотя бы одна падает с ошибкой — **отменяет остальные** и пробрасывает исключение в catch
- Родительская корутина (viewModelScope.launch) не крашится

Добавьте импорт:

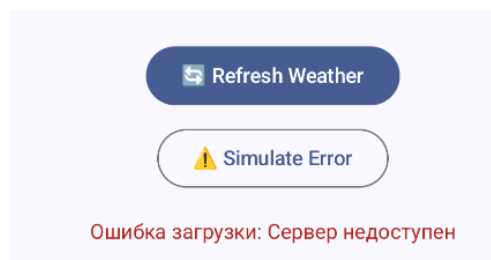
```
import kotlinx.coroutines.coroutineScope
```

11.5.4. Повторное тестирование

1. Запустите приложение заново
2. Нажмите "⚠ Simulate Error"
3. Нажмите "↺ Refresh Weather"

Что произойдёт теперь:

- Приложение **НЕ** крашнется!
- Появится сообщение красным цветом: **"Ошибка загрузки: Сервер недоступен"**



- Индикаторы загрузки остановятся
 - Кнопка "Refresh" снова станет активной
4. Нажмите "⚠ Simulate Error" снова (отключить имитацию ошибки)
 5. Нажмите "↺ Refresh Weather"

Ожидаемый результат:

- Загрузка проходит успешно
- Все три карточки заполняются данными
- Время загрузки ~2 секунды

11.5.5. Что мы узнали?

Проблема	Причина	Решение
Краш при ошибке в async	Исключение всплывает ДО await()	Использовать coroutineScope
catch не ловит ошибку	Ошибка в дочерней корутине	Обернуть код в coroutineScope
Logcat показывает "FATAL EXCEPTION"	Необработанное исключение	Читать stack trace, искать источник

Скриншоты для отчёта

- Скриншот Logcat с ошибкой FATAL EXCEPTION (первая попытка)
- Скриншот кода с исправлением (использование coroutineScope)
- Скриншот приложения с сообщением об ошибке (вторая попытка, без краша)
- Скриншот успешной загрузки после отключения ошибки
- Сохраните как:
 - **step11_5_logcat_error_FIO.png**
 - **step11_5_fixed_code_FIO.png**
 - **step11_5_error_handled_FIO.png**
 - **step11_5_success_after_error_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 11: Fix error handling in async coroutines"

git push

Шаг 12. Использование разных диспетчеров

12.1. Добавление тяжёлого вычисления

Что мы делаем: Покажем разницу между `Dispatchers.Main`, `Dispatchers.IO` и `Dispatchers.Default`.

Откройте `data/WeatherRepository.kt` и добавьте внутрь класса `WeatherRepository` новую функцию `calculateWeatherIndex()`, которая будет имитировать тяжёлые вычисления:

```
suspend fun calculateWeatherIndex(
    temp: Int,
    humidity: Int,
    wind: Int
): Int {
    // Это должно выполняться на Default, а не на Main!
    return withContext(Dispatchers.Default) {
        // Имитация сложных вычислений
        var result = 0
        for (i in 1..1000000) {
            result += (temp + humidity + wind) / 3
        }
        result / 1000000
    }
}
```

- `withContext(Dispatchers.Default)` — переключаемся на поток для вычислений
- Цикл из миллиона итераций — имитация тяжёлой работы
- Функция **suspend**, но внутри переключается на другой поток

Необходимые импорты:

```
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
```

12.2. Обновление WeatherData

Откройте `data/WeatherData.kt` и добавьте поле:

```
data class WeatherData(
    val temperature: Int? = null,
    val humidity: Int? = null,
    val windSpeed: Int? = null,
    +val weatherIndex: Int? = null,
```

12.3. Использование в ViewModel

Откройте `viewmodel/WeatherViewModel.kt` и измените метод `loadWeatherData()`.

Найдите блок внутри `coroutineScope`.

1. Добавим отображение загрузки:

```
coroutineScope {
    _weatherState.value = _weatherState.value.copy(
        loadingProgress = "Загружаем температуру, влажность, скорость ветра..."
    )
```

2. Добавим новый код для вычисления индекса:

```
val temperature = tempDeferred.await()
val humidity = humDeferred.await()
val windSpeed = windDeferred.await()

// Новый код: вычисление индекса
_weatherState.value = _weatherState.value.copy(
    loadingProgress = "Вычисление индекса погоды..."
)

+ val weatherIndex = repository.calculateWeatherIndex(
    temperature,
    humidity,
    windSpeed
)

_weatherState.value = WeatherData(
```

Также добавьте новое поле внутрь `_weatherState.value`:

```
_weatherState.value = WeatherData(
    temperature = temperature,
    humidity = humidity,
    windSpeed = windSpeed,
    + weatherIndex = weatherIndex, // Добавили!
    isLoading = false,
    error = null,
    loadingProgress = "Загрузка завершена!"
)
```

Объяснение:

- Вычисление происходит **после** загрузки данных
- `calculateWeatherIndex()` автоматически переключается на `Dispatchers.Default`
- **UI** не зависает, потому что вычисления не на главном потоке
- Всё это происходит внутри `coroutineScope`, поэтому ошибки обрабатываются корректно

12.4. Отображение индекса в UI

Откройте `MainActivity.kt` и добавьте внутри функции `WeatherDashboardScreen` после карточки `Wind Speed`:

```
WeatherCard(
    emoji = "☁️",
    title = "Wind Speed",
    value = weatherState.windSpeed?.let { "$it m/s" } ?: "-",
    isLoading = weatherState.isLoading && weatherState.windSpeed == null
)

if (weatherState.weatherIndex != null) {
    WeatherCard(
        emoji = "📊",
        title = "Weather Index",
        value = "${weatherState.weatherIndex}",
        isLoading = false
    )
}
}
```

Эмодзи для вставки: гистограмма - 📊

Объяснение:

- Показываем только если индекс вычислен
- **isLoading = false** — индикатор не нужен (показываем после загрузки всех данных)

12.5. Демонстрация диспетчеров

Добавьте комментарий в WeatherViewModel.kt перед loadWeatherData():

```
/**
 * Демонстрация работы диспетчеров:
 *
 * viewModelScope.launch - запускается на Dispatchers.Main
 *   ↳ coroutineScope { }
 *     ↳ async { fetchTemperature() } - выполняется на Dispatchers.IO (внутри repository)
 *     ↳ async { fetchHumidity() } - выполняется на Dispatchers.IO
 *     ↳ async { fetchWindSpeed() } - выполняется на Dispatchers.IO
 *     ↳ calculateWeatherIndex() - переключается на Dispatchers.Default
 *   ↳ обновление _weatherState - происходит на Dispatchers.Main
 *
 * Результат: UI никогда не блокируется!
 */
```

Итог:

```
> /** Демонстрация работы диспетчеров: ...*/
fun loadWeatherData() {
```

Таблица: Где что выполняется

Операция	Диспетчер	Почему?
viewModelScope.launch { }	Main	viewModelScope по умолчанию использует Main
coroutineScope { }	Main	Наследует диспетчер родителя
fetchTemperature()	IO	Имитация сетевого запроса (delay внутри)
calculateWeatherIndex()	Default	Тяжёлые вычисления (withContext внутри)
_weatherState.value = ...	Main	Обновление UI должно быть на Main

Важно:

- coroutineScope **не меняет** диспетчер, только группирует корутины
- withContext(Dispatchers.Default) внутри calculateWeatherIndex() **переключает** поток
- После выхода из withContext мы возвращаемся на Main

Скриншоты для отчёта

- Скриншот приложения с отображением Weather Index
- Скриншот кода с комментарием о диспетчерах
- Сохраните как:

- **step12_weather_index_FIO.png**
- **step12_dispatchers_comment_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 12: Add weather index calculation with Dispatchers.Default"

git push

Шаг 13. Добавление таймера обновления

13.1. Автоматическое обновление каждые 10 секунд

Используем **Flow** для автоматического обновления данных.

Откройте **viewmodel/WeatherViewModel.kt** и добавьте импорты:

```
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.flow
```

Добавьте функцию в класс **WeatherViewModel**:

```
private fun startAutoRefresh() {
    viewModelScope.launch {
        // Бесконечный поток с задержкой
        flow {
            while (true) {
                delay(10000) // 10 секунд
                emit(Unit)
            }
        }.collect {
            loadWeatherData()
        }
    }
}
```

- **flow { }** — создаём поток данных
- **while (true)** — бесконечный цикл
- **delay(10000)** — пауза 10 секунд (не блокирует поток!)
- **emit(Unit)** — отправляем сигнал в поток
- **.collect { }** — подписываемся на поток
- Каждые 10 секунд вызывается **loadWeatherData()**

Вызовите в **init**:

```
init {  
    loadWeatherData()  
    +startAutoRefresh()  
}
```

13.2. Отмена автообновления

Проблема: Автообновление будет работать даже после выхода из приложения!

Решение: `viewModelScope` автоматически отменяет все корутины при уничтожении `ViewModel`.

Добавьте комментарий:

```
init {  
    loadWeatherData()  
    startAutoRefresh()  
    +// viewModelScope автоматически отменит корутину при onCleared()  
}
```

Как это работает:

User открывает экран → `ViewModel` создается → `init` вызывается

↓

`startAutoRefresh()` запускается

↓

`Flow` начинает работу (каждые 10 сек)

↓

User закрывает экран → `ViewModel.onCleared()` вызывается

↓

`viewModelScope` отменяется

↓

`Flow` останавливается автоматически

13.3. Индикатор автообновления

Добавьте в `MainActivity.kt` внутри функции `WeatherDashboardScreen()` после карточки `Weather Index`:

```

    if (weatherState.weatherIndex != null) {...}
    Spacer(modifier = Modifier.height(16.dp))
    Row(
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "🔄 Auto-refresh every 10s",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.tertiary
        )
    }

```

- Row — горизонтальный контейнер
- bodySmall — маленький шрифт
- tertiary — третичный цвет (неяркий, ненавязчивый)

13.4. Тестирование автообновления

1. Запустите приложение
2. Дождитесь первой загрузки данных
3. Запомните значения (например, температура = 25°C)
4. Подождите 10 секунд
5. Заметьте, что данные обновились (температура изменилась)

Что происходит:

- Каждые 10 секунд loadWeatherData() вызывается автоматически
- Данные загружаются заново (с новыми случайными значениями)
- UI обновляется плавно

Проверка отмены:

1. Нажмите кнопку "Back" (выход из приложения)
2. Корутина автоматически остановится
3. Приложение не будет потреблять ресурсы в фоне

Скриншоты для отчёта

- Скриншот кода startAutoRefresh()
- Скриншот приложения с индикатором автообновления
- Скриншот приложения через 10 секунд (новые данные)

- Сохраните как:
- **step13_auto_refresh_code_FIO.png**
- **step13_auto_refresh_ui_FIO.png**
- **step13_refreshed_data_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 13: Add auto-refresh every 10 seconds"

git push

Шаг 14. Создание README.md

Создайте файл README.md в корне проекта со следующей информацией:

Файл должен содержать:

1. Название проекта
2. Краткое описание (2-3 предложения)
3. Функциональность приложения (список из 5-7 пунктов)
4. Технологии и библиотеки (список)
5. Ответьте на контрольные вопросы:

Контрольные вопросы

A) В чём разница между launch и async?

- Когда использовать каждый из них?
- Приведите пример кода

B) Что такое suspend функция?

- Может ли она вызываться из обычной функции?
- Почему delay() не блокирует поток?

C) Зачем нужны разные диспетчеры?

- Приведите таблицу с 3 диспетчерами и примерами задач
- Что будет, если выполнить тяжёлое вычисление на Dispatchers.Main?

D) Что произойдёт, если не обработать исключение в корутине?

- Как корректно обрабатывать ошибки?
- Зачем нужен try-catch внутри launch?

E) Как работает автоматическая отмена корутин?

- Что такое viewModelScope?

- Когда корутины отменяются автоматически?

6. Как запустить проект (2-3 шага)

7. Скриншоты работы приложения

8. Автор и дата выполнения

После выполните в терминале:

git add .

git commit -m "Step 14: Add project README"

git push

Самостоятельная работа

Создание собственной иконки

В лабораторной работе №8 мы создавали собственную иконку. Вам нужно для текущего приложения создать аналогично свою.

1. Создайте уникальную иконку в стилистике вашей темы
2. Используйте Krita, Canva, Figma или другое приложение
3. Добавьте через **Image Asset Studio**